

Investigation of Machine Learning Methods for Financial Fraud Detection

Author: Amirhossein Ranjbar

Abstract

Detecting fraudulent financial transactions is an important problem in the field of machine learning. Data imbalance, the very small number of fraudulent samples, and the high cost of incorrect predictions are among the main challenges of this problem.

In this report, several baseline models, including Decision Tree, KNN, and Logistic Regression, are first investigated. More advanced models, including XGBoost and Random Forest, are then evaluated. In addition, different models are combined using an Ensemble approach.

The performance of the models is evaluated using metrics such as Precision, Recall, *F1-score*, and PR-AUC. Cross-validation and hyperparameter optimization methods are also used to select appropriate model configurations.

Keywords:

Machine Learning, Fraud Detection, Financial Transactions, Decision Tree, KNN, Logistic Regression, Random Forest, XGBoost, Ensemble, Imbalanced Data

1. Introduction

The increasing use of electronic payment systems has resulted in a very large number of financial transactions being recorded every day. Along with this growth, there is also a possibility of fraudulent transactions.

Automatic detection of fraudulent transactions can play an important role in reducing financial losses. However, fraud detection is usually associated with the problem of class imbalance.

In many real-world datasets, the number of legitimate transactions is much greater than the number of fraudulent transactions. Therefore, a model may achieve very high accuracy while still performing poorly in detecting fraudulent samples.

For this reason, metrics such as Precision, Recall, *F1-score*, and PR-AUC are particularly important in this problem.

In this study, baseline models are first investigated, followed by more advanced models and, finally, an Ensemble method.

2. Dataset

In this study, a dataset containing financial transactions is used. Each sample contains several transaction-related features and a binary label.

A label of zero represents a legitimate transaction, while a label of one represents a fraudulent transaction.

One of the important characteristics of the dataset is the severe imbalance between the two classes. Therefore, using Accuracy alone is not an appropriate evaluation criterion.

After preprocessing, the data were divided into training and test sets.

3. Data Preprocessing

Before training the models, the data were examined and prepared.

The preprocessing steps included checking for missing values, duplicate records, and preparing the features for model input.

As observed during the initial data analysis, the dataset contained a number of duplicate records. Since duplicate samples can affect the model training process, these samples were identified and removed.

The number of samples and fraudulent transactions before and after duplicate removal is presented in Table ??.

Table 1: Comparison of the dataset before and after duplicate removal

Data	Before	After
Total transactions	284807	284315
Fraudulent transactions	492	473

As shown, the total number of transactions decreased from 284807 to 284315. The number of fraudulent transactions also decreased from 492 to 473 samples.

The data were divided into training and test sets:

$$D = D_{\text{train}} \cup D_{\text{test}}$$

where the test set is not used during the model training process.

When features have different scales, standardization methods can be used to prepare the data. Standardization is particularly important for models such as KNN and Logistic Regression.

4. Baseline Models

In the first stage of the study, three baseline models were investigated to establish an initial performance benchmark:

- Decision Tree
- K-Nearest Neighbors (KNN)
- Logistic Regression

The purpose of using baseline models was to evaluate the performance of different algorithms and establish a basis for comparison with more advanced models.

4.1. Decision Tree

Decision Tree is a machine learning algorithm that creates a tree-based structure for prediction by repeatedly splitting the data according to different features.

The complexity and performance of the model depend on several hyperparameters. In this experiment, three important hyperparameters, including *max_depth*, *min_samples_split*, and *min_samples_leaf*, were investigated.

The *max_depth* parameter specifies the maximum depth of the tree. Increasing this value allows more complex structures to be created, but excessive increases may lead to overfitting.

For this parameter, the values 3, 5, 8, 10, 15, 20, and *None* were investigated.

The *min_samples_split* parameter specifies the minimum number of samples required to split a node. The values 2, 5, 10, and 20 were considered.

The *min_samples_leaf* parameter specifies the minimum number of samples that must be present in each leaf node. The values 1, 2, 5, and 10 were evaluated.

Grid Search was used to select the best combination of these hyperparameters.

Since 7, 4, and 4 values were considered for the three hyperparameters, respectively, the total number of combinations was:

$$7 \times 4 \times 4 = 112$$

For evaluating each combination, Stratified K-Fold Cross Validation with five folds was used. The *F1-score* was used as the selection criterion.

The final selected hyperparameters were:

Table 2: Selected hyperparameters for Decision Tree

Hyperparameter	Evaluated Values	Selected Value
<i>max_depth</i>	3, 5, 8, 10, 15, 20, <i>None</i>	8
<i>min_samples_split</i>	2, 5, 10, 20	20
<i>min_samples_leaf</i>	1, 2, 5, 10	5

After selecting the hyperparameters, the final model was trained using the training data and evaluated on the test set.

Table 3: Decision Tree performance on the test set

Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56651
1	0.92	0.71	0.80	95
Macro Avg	0.96	0.85	0.90	56746
Weighted Avg	1.00	1.00	1.00	56746

The confusion matrix of the model is presented below:

Table 4: Decision Tree confusion matrix

	Predicted Class 0	Predicted Class 1
Actual Class 0	56645	6
Actual Class 1	28	67

According to the results, the model achieved a Precision of 0.92 and a Recall of 0.71 for the minority class.

The model correctly identified 67 out of the 95 positive samples, while 28 positive samples were incorrectly classified as class 0.

Only 6 samples from class 0 were incorrectly classified as class 1. The resulting *F1-score* was 0.80.

4.2. K-Nearest Neighbors

K-Nearest Neighbors (KNN) is a distance-based algorithm that classifies a new sample by examining its nearest samples in the training dataset.

The *n_neighbors*, or *K*, parameter specifies the number of neighbors used in the decision-making process.

Since KNN operates based on distances between samples, feature standardization is particularly important. In this study, the training features were standardized before being used by KNN.

Grid Search combined with Stratified K-Fold Cross Validation was used to select the best model configuration.

Three hyperparameters, including *n_neighbors*, *weights*, and *p*, were investigated.

For *n_neighbors*, the values 3, 5, 7, 9, 11, 15, 20, 25, and 30 were considered.

For the *weights* parameter, the two options *uniform* and *distance* were investigated. With *distance*, closer samples receive greater weights in the decision-making process.

The *p* parameter specifies the type of distance metric. A value of *p* = 1 corresponds to Manhattan distance, while *p* = 2 corresponds to Euclidean distance.

The total number of combinations was:

$$9 \times 2 \times 2 = 36$$

Stratified K-Fold Cross Validation with five folds was used to evaluate each combination, and *F1-score* was used as the selection criterion.

Table 5: Selected hyperparameters for KNN

Hyperparameter	Evaluated Values	Selected Value
<i>n_neighbors</i>	3, 5, 7, 9, 11, 15, 20, 25, 30	3
<i>weights</i>	<i>uniform</i> , <i>distance</i>	<i>distance</i>
<i>p</i>	1, 2	1

Therefore, the best KNN configuration consisted of *K* = 3, distance weighting, and Manhattan distance (*p* = 1).

The best validation *F1-score* was 0.8543.

After selecting the hyperparameters, the final model was trained using the standardized training data and evaluated on the test set.

Table 6: KNN performance on the test set

Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56651
1	0.96	0.72	0.82	95
Macro Avg	0.98	0.86	0.91	56746
Weighted Avg	1.00	1.00	1.00	56746

The confusion matrix is presented below:

Table 7: KNN confusion matrix

	Predicted Class 0	Predicted Class 1
Actual Class 0	56648	3
Actual Class 1	27	68

According to the results, KNN achieved a Precision of 0.96 and a Recall of 0.72 for the minority class.

The model correctly identified 68 out of 95 fraudulent transactions, while 27 fraudulent samples were missed.

Only 3 legitimate transactions were incorrectly classified as fraudulent. The resulting *F1-score* was 0.82.

4.3. Logistic Regression

Logistic Regression is a classical method for binary classification problems. It calculates a linear combination of the features and uses the sigmoid function to obtain the probability of a sample belonging to the positive class.

The sigmoid function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where:

$$z = w^T x + b$$

Since Logistic Regression is sensitive to feature scales, the features were standardized using *StandardScaler*.

Grid Search combined with Stratified K-Fold Cross Validation was used to select the best configuration.

Two hyperparameters, *C* and *class_weight*, were investigated.

The *C* parameter represents the inverse of the regularization strength. A smaller value of *C* corresponds to stronger regularization, while a larger value corresponds to weaker regularization.

The values 0.01, 0.1, 1, 10, and 100 were considered.

For *class_weight*, the two options *None* and *balanced* were investigated.

The total number of combinations was:

$$5 \times 2 = 10$$

Stratified K-Fold Cross Validation with five folds was used for evaluation, and *F1-score* was used as the selection criterion.

Table 8: Selected hyperparameters for Logistic Regression

Hyperparameter	Evaluated Values	Selected Value
<i>C</i>	0.01, 0.1, 1, 10, 100	10
<i>class_weight</i>	<i>None</i> , <i>balanced</i>	<i>None</i>

Finally, the configuration with *C=10* and *class_weight=None* was selected.

The best validation *F1-score* was 0.7118.

After selecting the hyperparameters, the final model was trained using the standardized training data and evaluated on the test set.

Table 9: Logistic Regression performance on the test set

Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56651
1	0.85	0.58	0.69	95
Macro Avg	0.92	0.79	0.84	56746
Weighted Avg	1.00	1.00	1.00	56746

According to the results, Logistic Regression achieved a Precision of 0.85 and a Recall of 0.58 for the minority class.

The *F1-score* for the fraud class was 0.69. This result indicates that, on this dataset, Logistic Regression performed worse than nonlinear models such as Decision Tree and KNN in detecting the minority class.

4.4. Summary of Baseline Models

In the first stage, three baseline models, including Decision Tree, KNN, and Logistic Regression, were investigated.

Grid Search combined with five-fold Stratified K-Fold Cross Validation and *F1-score* as the evaluation criterion was used to select the appropriate hyperparameters for each model.

The results showed that KNN achieved the best performance among the baseline models, with an *F1-score* of 0.82. This model achieved a Precision of 0.96 and a Recall of 0.72 for the fraud class using *K* = 3, distance weighting, and Manhattan distance.

Decision Tree achieved a Precision of 0.92, Recall of 0.71, and *F1-score* of 0.80, showing performance close to KNN.

In contrast, Logistic Regression achieved a Precision of 0.85, Recall of 0.58, and *F1-score* of 0.69, making it the weakest model for detecting fraudulent transactions.

Therefore, among the baseline models, KNN achieved the best performance, while Logistic Regression showed the weakest performance.

The results also demonstrate that although all three models achieved very high Accuracy, their minority-class metrics differed considerably. Therefore, Precision, Recall, and *F1-score* are more appropriate metrics for comparing the models in this problem.

Considering the limitations of the baseline models and the nonlinear and imbalanced nature of the dataset, more powerful models such as XGBoost and Random Forest were considered.

The objective of this stage was to investigate whether better performance could be achieved in detecting the minority class and ultimately use these models in an Ensemble method.

5. Advanced Models

After evaluating the baseline models, more powerful tree-based models were investigated.

The following models were used in this stage:

- XGBoost
- Random Forest
- KNN

KNN was also retained as one of the models used in the Ensemble to investigate its combination with tree-based models.

5.1. XGBoost

XGBoost is a powerful algorithm based on Gradient Boosting that uses a collection of decision trees to construct a final model.

Unlike methods such as Random Forest, where trees are trained independently, XGBoost builds trees sequen-

tially. Each new tree attempts to reduce the errors made by the previous models.

The general form of the XGBoost model can be represented as:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i)$$

where K is the number of trees and f_k is the function corresponding to the k -th decision tree.

The XGBoost objective function consists of a loss function and a regularization term:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

The regularization term helps control model complexity and reduce the risk of overfitting.

Since the dataset has a severe imbalance between legitimate and fraudulent classes, the *scale_pos_weight* parameter was used.

This parameter was calculated using the ratio of the number of legitimate samples to fraudulent samples:

$$\text{scale_pos_weight} = \frac{N_0}{N_1}$$

where N_0 represents the number of legitimate samples and N_1 represents the number of fraudulent samples.

Therefore, minority-class samples receive greater weight during the learning process.

Grid Search combined with five-fold Stratified K-Fold Cross Validation was used to select the best XGBoost configuration.

The *F1-score* was used as the model selection criterion.

Six hyperparameters were investigated:

- *n_estimators*: 300 and 500
- *max_depth*: 3, 4, and 5
- *learning_rate*: 0.03 and 0.05
- *min_child_weight*: 1 and 3
- *subsample*: 0.8 and 1.0
- *colsample_bytree*: 0.8 and 1.0

Therefore, the total number of combinations was:

$$2 \times 3 \times 2 \times 2 \times 2 \times 2 = 96$$

Since five folds were used in Cross-Validation, a total of:

$$96 \times 5 = 480$$

training and evaluation runs were performed. The best hyperparameter configuration was:

Table 10: Selected hyperparameters for XGBoost

Hyperparameter	Selected Value
<i>n_estimators</i>	500
<i>max_depth</i>	5
<i>learning_rate</i>	0.05
<i>min_child_weight</i>	1
<i>subsample</i>	0.8
<i>colsample_bytree</i>	1.0

The best Cross-Validation *F1-score* was 0.8639.

This value indicates that XGBoost achieved a good balance between Precision and Recall for the minority class during validation.

After selecting the best hyperparameters, the final model was trained using the complete training data and evaluated on the test set.

Table 11: XGBoost performance on the test set

Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56651
1	0.93	0.79	0.85	95
Macro Avg	0.96	0.89	0.93	56746
Weighted Avg	1.00	1.00	1.00	56746

According to the results, XGBoost achieved a Precision of 0.93, Recall of 0.79, and *F1-score* of 0.85 for the minority class.

Therefore, the model successfully detected approximately 79% of the fraudulent transactions in the test set.

The Precision of 0.93 indicates that most of the transactions identified by the model as fraudulent were actually fraudulent.

The *F1-score* of 0.85 also indicates a good balance between Precision and Recall for the fraud class.

Since *F1-score* was used as the model selection criterion in Grid Search, the validation result was 0.8639, while the test result was 0.85.

The relatively close values indicate that the performance drop from validation to the test set was limited.

Compared with the baseline models, XGBoost achieved better performance in detecting the minority class. Specifically, its fraud-class *F1-score* of 0.85 was higher than Decision Tree with 0.80 and Logistic Regression with 0.69, and it also outperformed KNN with 0.82.

The Precision of 0.93 combined with a Recall of 0.79 indicates that XGBoost is a suitable candidate for the next stage, namely combining models using an Ensemble approach.

5.2. Random Forest

Random Forest consists of a large number of decision trees. Each tree is trained using different samples and features, and the outputs of the trees are combined to produce the final prediction.

The probability of a sample belonging to the fraud class can be represented as:

$$P(y = 1|x) = \frac{1}{T} \sum_{t=1}^T P_t(y = 1|x)$$

where T represents the number of trees.

An important advantage of Random Forest is the reduction of variance and the lower risk of overfitting compared with a single decision tree.

In this study, parameters such as the number of trees, *max_depth*, *min_samples_split*, *min_samples_leaf*, and *max_features* were investigated.

Stratified K-Fold Cross-Validation and *F1-score* were used to select the appropriate configuration.

Due to the severe imbalance between the legitimate and fraud classes, *class_weight=balanced* was used.

Grid Search combined with five-fold Stratified K-Fold Cross Validation was used to select the best Random Forest configuration.

The *F1-score* was used as the selection criterion.

Five hyperparameters were investigated:

- *n_estimators*: 200 and 400
- *max_depth*: 10, 11, 12, 13, and 14
- *min_samples_split*: 2 and 5
- *min_samples_leaf*: 1 and 2
- *max_features*: *sqrt*

Therefore, the total number of combinations was:

$$2 \times 5 \times 2 \times 2 \times 1 = 40$$

Since five folds were used in Cross-Validation, a total of:

$$40 \times 5 = 200$$

training and evaluation runs were performed.

The best hyperparameter configuration was:

Table 12: Selected hyperparameters for Random Forest

Hyperparameter	Selected Value
<i>n_estimators</i>	200
<i>max_depth</i>	13
<i>min_samples_split</i>	2
<i>min_samples_leaf</i>	2
<i>max_features</i>	<i>sqrt</i>

The best Cross-Validation *F1-score* was 0.8495.

After selecting the best hyperparameters, the final model was trained using the complete training data and evaluated on the test set.

Table 13: Random Forest performance on the test set

Class	Precision	Recall	F1-score	Support
0	1.00	1.00	1.00	56651
1	0.92	0.73	0.81	95
Macro Avg	0.96	0.86	0.91	56746
Weighted Avg	1.00	1.00	1.00	56746

According to the results, Random Forest achieved a Precision of 0.92, Recall of 0.73, and *F1-score* of 0.81 for the minority class.

The model successfully detected approximately 73% of the fraudulent transactions in the test set.

The Precision of 0.92 indicates that the model has good accuracy in identifying fraudulent transactions.

The *F1-score* of 0.81 also indicates a reasonable balance between Precision and Recall for the minority class.

Comparing the Cross-Validation *F1-score* with the test result shows that the metric decreased from 0.8495 during validation to 0.81 on the test set.

This difference indicates that the model performed slightly worse on the test data than its average performance during Cross-Validation.

Compared with XGBoost, Random Forest achieved weaker performance in detecting fraudulent transactions.

XGBoost achieved Precision of 0.93, Recall of 0.79, and *F1-score* of 0.85, while Random Forest achieved 0.92, 0.73, and 0.81, respectively.

Nevertheless, Random Forest still provides reasonable performance and, due to its different learning structure compared with XGBoost, can be used as one of the models in the Ensemble stage.

5.3. Ensemble Weight Selection Using OOF

After training XGBoost, Random Forest, and KNN, the objective of this stage was to select appropriate weights for combining the three models.

To avoid selecting weights based on predictions generated from samples that the models had already seen during training, Out-of-Fold Predictions, or OOF predictions, were used.

Stratified K-Fold Cross Validation with five folds was used.

In each fold, the models were trained on four folds and predictions were generated for the remaining fold.

Therefore, for each training sample, a predicted probability was obtained from a model that had not seen that sample during training.

For all three models, the probability of belonging to the fraud class was stored:

$$P_{XGB}^{OOF}, \quad P_{RF}^{OOF}, \quad P_{KNN}^{OOF}$$

The final Ensemble probability was then calculated as a weighted combination of the three model probabilities:

$$P_{ensemble} = w_{XGB}P_{XGB} + w_{RF}P_{RF} + w_{KNN}P_{KNN}$$

where the sum of the weights is equal to one:

$$w_{XGB} + w_{RF} + w_{KNN} = 1$$

To select the weights, the XGBoost and Random Forest weights were evaluated with a step size of 0.05, while the KNN weight was calculated as:

$$w_{KNN} = 1 - w_{XGB} - w_{RF}$$

Different thresholds from 0.05 to 0.95 with a step size of 0.01 were also evaluated.

For each combination of weights and threshold, Precision, Recall, and *F1-score* were calculated.

The main objective was to select a combination satisfying:

$$\text{Precision} \geq 0.90$$

and

$$\text{Recall} \geq 0.80$$

Among the combinations satisfying both conditions, the combination with the highest *F1-score* was selected.

If no combination could simultaneously satisfy both conditions, the combination with the highest *F1-score* among all evaluated combinations was selected.

The weight and threshold selection process was performed algorithmically, and OOF predictions for all three models were generated for all training samples.

After completing the search process, the final weights and decision threshold were:

Table 14: Selected parameters for the Ensemble

Parameter	Value
XGBoost weight	0.45
Random Forest weight	0.45
KNN weight	0.10
Decision threshold	0.56
<i>F1-score</i> on OOF	0.8727

The final values in the table were obtained directly from the OOF process and the weight and threshold search.

An important point is that the test set was not used to select the weights or threshold. Therefore, the test set remained an independent dataset for the final evaluation of the Ensemble.

5.4. Ensemble Results

In the final stage, the probabilities generated by XGBoost, Random Forest, and KNN were combined using a weighted Ensemble.

OOF predictions obtained from five-fold Stratified K-Fold Cross Validation were used to select the appropriate weights.

During the search process, the XGBoost and Random Forest weights were evaluated with a step size of 0.05, while the KNN weight was determined so that the sum of all three weights was equal to one.

Decision thresholds between 0.05 and 0.95 with a step size of 0.01 were also evaluated.

The objective was to select a combination that, if possible, achieved a Precision of at least 0.90 and a Recall of at least 0.80, while maximizing the *F1-score* among the valid combinations.

The final weights selected based on OOF predictions were:

$$w_{\text{XGB}} = 0.45$$

$$w_{\text{RF}} = 0.45$$

$$w_{\text{KNN}} = 0.10$$

The decision threshold was:

$$\tau = 0.56$$

Therefore, the final Ensemble probability is calculated as:

$$P_{\text{ensemble}} = 0.45P_{\text{XGB}} + 0.45P_{\text{RF}} + 0.10P_{\text{KNN}}$$

The final class label is determined using the threshold $\tau = 0.56$:

$$\hat{y} = \begin{cases} 1 & P_{\text{ensemble}} \geq 0.56 \\ 0 & P_{\text{ensemble}} < 0.56 \end{cases}$$

The *F1-score* obtained from the OOF predictions was 0.8727.

The final Ensemble results on the test set were:

Table 15: Final Ensemble results on the test set

Parameter	Value
XGBoost weight	0.45
Random Forest weight	0.45
KNN weight	0.10
Threshold	0.56
Precision	0.9730
Recall	0.7579
<i>F1-score</i>	0.8521
PR-AUC	0.8253

As shown, the Ensemble model achieved a Precision of 0.9730 on the test set.

The Recall was 0.7579. Therefore, the model correctly identified 72 out of the 95 fraudulent transactions in the test set.

The resulting *F1-score* was 0.8521, indicating a good balance between Precision and Recall.

The confusion matrix of the Ensemble model was:

Table 16: Ensemble confusion matrix

	Predicted Class 0	Predicted Class 1
Actual Class 0	56649	2
Actual Class 1	23	72

According to the confusion matrix, the model incorrectly classified only 2 legitimate samples as fraudulent.

In contrast, out of the 95 fraudulent samples, 72 were correctly identified and 23 were missed.

The PR-AUC of the model was 0.8253.

Considering the severe class imbalance, this metric is an appropriate criterion for evaluating the performance of the model in detecting the minority class.

Compared with the individual models, the Ensemble achieved a very high Precision. For example, XGBoost alone achieved Precision of 0.93, Recall of 0.79, and *F1-score* of 0.85, whereas the Ensemble achieved Precision of 0.97 and *F1-score* of 0.85.

However, the Ensemble Recall was 0.76, which was lower than the Recall of 0.79 achieved by XGBoost.

Therefore, in this experiment, combining the models increased Precision and reduced the number of false positives, but this came at the cost of a relative decrease in Recall.

The OOF *F1-score* was 0.8727, while the test-set *F1-score* was 0.8521.

The limited decrease from OOF to the test set indicates that the model maintained a reasonable level of performance on unseen data.

Since the initial objective was to select a combination with Precision of at least 0.90 and Recall of at least 0.80, the selected combination was unable to maintain the Recall requirement of 0.80 on the test set.

Therefore, although the combination achieved strong performance on the training data and OOF predictions, its Recall decreased to approximately 0.76 on the test set.

6. Overall Results and Comparative Analysis

This section compares the performance of all investigated models.

Due to the severe imbalance in the dataset, the primary focus is on the ability of the models to detect the minority class, namely fraudulent transactions.

Therefore, Precision, Recall, and *F1-score* are more important than Accuracy.

The final results are summarized in the following table:

Table 17: Comparison of model performance on the test set

Model	Precision	Recall	F1-score
Decision Tree	0.92	0.71	0.80
KNN	0.96	0.72	0.82
Logistic Regression	0.85	0.58	0.69
Random Forest	0.92	0.73	0.81
XGBoost	0.93	0.79	0.85
Ensemble	0.97	0.76	0.85

Based on the results, Logistic Regression achieved the weakest performance among the evaluated models, with an *F1-score* of 0.69.

Its Recall of 0.58 indicates that the model missed a considerable portion of fraudulent transactions.

This result suggests that linear models have less ability to capture the complex nonlinear relationships present in the dataset.

Among the baseline models, KNN achieved the best performance with an *F1-score* of 0.82.

The model also achieved a Precision of 0.96, indicating that the number of legitimate transactions incorrectly classified as fraudulent was relatively low.

Random Forest achieved an *F1-score* of 0.81, which was close to the performance of KNN.

However, its Recall of 0.73 indicates that it missed more fraudulent transactions than XGBoost.

The best performance among the individual models was achieved by XGBoost.

This model achieved Precision of 0.93, Recall of 0.79, and *F1-score* of 0.85, providing the best balance between correctly identifying fraudulent transactions and avoiding false positives.

The comparison between XGBoost and Random Forest also shows that XGBoost performed better across all three main metrics.

Recall increased from 0.73 in Random Forest to 0.79 in XGBoost, while *F1-score* increased from 0.81 to 0.85.

This result suggests that the sequential learning process of Boosting in XGBoost was able to model more complex relationships in this dataset compared with the Bagging approach of Random Forest.

In the final stage, XGBoost, Random Forest, and KNN were combined using a weighted Ensemble.

The selected weights were 0.45 for XGBoost, 0.45 for Random Forest, and 0.10 for KNN.

The decision threshold was also set to 0.56.

The Ensemble increased Precision from 0.93 for XGBoost to 0.97.

Furthermore, the number of false positives on the test set was only 2.

This indicates that the combination of models was highly effective in reducing false positives.

However, the increase in Precision was accompanied by a decrease in Recall.

The Ensemble Recall was 0.76, whereas XGBoost achieved a Recall of 0.79.

Therefore, in this experiment, the Ensemble did not outperform XGBoost in terms of identifying a larger number of fraudulent transactions.

The *F1-score* of both XGBoost and Ensemble was approximately 0.85.

Therefore, based on the *F1-score*, combining the models did not provide a significant improvement over XGBoost alone.

However, the Ensemble provides a greater advantage when reducing false positives and increasing Precision are more important.

Another important result is the difference between OOF performance and test-set performance.

The Ensemble *F1-score* on OOF predictions was 0.8727, while it decreased to 0.8521 on the test set.

This decrease indicates that the model performed slightly worse on completely unseen data than during the weight selection process.

Overall, the results show that the best model depends on the objective of the fraud detection system.

If the primary objective is to achieve a balance between Precision and Recall while detecting a larger number of fraudulent transactions, XGBoost is the best option in this experiment.

On the other hand, if reducing false positives is more important, the Ensemble, with a Precision of 0.97, is a more suitable option.

Therefore, based on the test-set results, XGBoost can be considered the best individual model in this study, while the Ensemble demonstrates that combining models does not necessarily guarantee simultaneous improvements in all evaluation metrics.

The selection of model weights and the decision

threshold plays an important role in determining the trade-off between Precision and Recall.

References

- [1] J. Han, M. Kamber, and J. Pei,
Data Mining: Concepts and Techniques.
- [2] I. Goodfellow, Y. Bengio, and A. Courville,
Deep Learning,
MIT Press, 2016.
- [3] C. M. Bishop,
Pattern Recognition and Machine Learning,
Springer, 2006.